



Université de Sherbrooke

SQL

Expressions simples

SQL_03

Christina KHNAISSER (christina.khnaisser@usherbrooke.ca)

Luc LAVOIE (luc.lavoie@usherbrooke.ca)

Ameni SOUID (ameni.Souid@usherbrooke.ca)

Scriptorum/Scriptorum/SQL_03a-Expressions-simples, version 1.0.1.a, en date du 2026-01-29

— document en cours de révision, ne pas citer à l'externe —

Plan

Introduction	3
1. Présentation	4
2. Projection-Extension-Renommage	6
3. Restriction	20
4. Jointure	27
5. Opérations complémentaires	47
Conclusion	54
Références	55
Définitions	56

Introduction

La séquence d'opérations jointure-restriction-projection-extension contribue en pratique à une proportion importante des requêtes, dans la mesure d'un renommage occasionnel des termes. En réunissant cette séquence dans une même instruction, les concepteurs du langage SQL voulaient donner une allure procédurale au langage, jugeant qu'elle faciliterait l'adhésion des programmeurs; offrir un raccourci notationnel aux programmeurs; faciliter l'optimisation conjointe des opérations. Les opérateurs ensemblistes (union, différence, intersection) seront intégrés par la suite, d'une façon moins commode. Au fil du temps, plusieurs opérateurs (ou pseudo-opérateurs) seront ajoutés – certains d'entre eux seront couverts dans le module suivant.

1. Présentation

Restriction
 $R \sigma \text{ cond}$

A	B	C
a1	b1	c1
a2	b2	c2
a3	b3	c3

Projection
 $R \pi \{A, C\}$

A	B	C
a1	b1	c1
a2	b2	c2
a3	b3	c3

Jointure naturelle
 $R \bowtie S$

A	B	B	C
a1	b1	b1	c1
a2	b1	b2	c2
a3	b3	b3	c3
a4	b4	b3	c4

**Extension
(augmentation)**
 $R \alpha C:f$

A	B	A	B	C
a1	b1	a1	b1	f(a1,b1)
a2	b1	a2	b1	f(a2,b1)
a3	b3	a3	b3	f(a3,b3)
a4	b4	a4	b4	f(a4,b4)

Renommage
 $R \rho A:C$

A	C
...	...

Une requête n'est pas autre chose qu'une expression SELECT.

```
requête ::=  
  [ Contexte ]  
  SELECT opMode { * | projection-extension }  
  FROM listeDeJointures  
  [ Restriction ]  
  [ Groupement ]  
  [ OpComplémentaire ]  
  [ Ordonnancement ]  
  [ Divers ]
```

2. Projection-Extension-Renommage

Définition de l'entête du résultat.

```
SELECT opMode { * | projection-extension }  
FROM listeDeJointures
```

```
opMode ::=  
    [ DISTINCT | ALL ]
```

```
projection-extension ::=  
    { expression [ [ AS ] nomColonne ] ... , }
```

```
listeDeJointures ::=  
    nomTable
```

Le mode indique si l'expression retourne une relation (DISTINCT) ou une collection (ALL).

L'étoile indique que tous les attributs de l'expression sont retenus (sans projection).

Exemple — Université

- Quels sont les types d'évaluation ?

```
select *  
from TypeEvaluation;
```

- Quels sont les matricules des personnes étudiantes admises ?

Clarification: on suppose ici que Etudiant donne la liste des personnes étudiantes ayant été admises.

```
select matricule  
from Etudiant;
```

2.1. Opérateur CASE

C'est un opérateur de choix.

```
<simple case> ::=  
    CASE WHEN <condition> THEN <result>  
    [ ELSE <result> ]  
    END
```

```
<result> ::=  
    <expression> | NULL
```

Exemple — Université

- Quelles sont les personnes étudiantes inscrites ?

Donner le matricule et le nom du département.

Clarification: on suppose ici qu'une personne étudiante n'est inscrite qu'à partir du moment où elle a un résultat d'évaluation.

```
select matricule
      , case when substr(activite,1, 3) in ('IFT', 'IMN', 'IGE')
then 'Informatique'
      when substr(activite,1, 3) in ('GMQ') then 'Géomatique'
      end as departement
from Resultat;
```

- Étendre la table résultat pour ajouter :
la cote 'A' si la note est au moins de 90 et
la cote 'B' si la note au moins de 70 (mais inférieure à 90).

```
select matricule, activite  
      , case  
        when note >= 90 then 'A'  
        when note between 70 and 89 then 'B'  
        -- else 'C'  
      end as cote  
from Resultat;
```

2.2. Fonctions d'aggrégations

- AVG(col)
- COUNT(col)
- MAX(col)
- MIN(col)
- SUM(col)
- MEDIAN(col)

Exemple — Université

- Combien y-a-t'il d'activités offertes ?

```
select count(*) as nbActivites  
from Activite;
```

- Calculer la moyenne, le minimum et le maximum des résultats;

```
select avg(note) as moy_exa  
      , min(note) as min_exa  
      , max(note) as max_exa  
from Resultat.
```

3. Restriction

Définition de l'ensemble de tuples à retenir.

```
requête ::=  
  [ Contexte ]  
  SELECT opMode { * | projection-extension }  
  FROM listeDeJointures  
  WHERE condition  
  [ Groupement ]  
  [ OpComplémentaire ]  
  [ Ordonnancement ]  
  [ Divers ]
```

Exemple — Université

- Quelles sont les activités offertes au département d'informatique ?

```
select *  
from activite  
where substr(sigle,1, 3) in ('IFT', 'IMN', 'IGE');
```

- Quelles sont les personnes étudiantes inscrites à l'activité IFT187 ?

```
select matricule  
from Resultat  
where activite = 'IFT187';
```

- Quelles sont les personnes étudiantes inscrites à une activité à l'automne ?

```
select matricule  
from Resultat  
where substr(trimestre, 5) = '3';
```

- Calculer la moyenne, le minimum et le maximum des résultats des examens (IN, FI) de l'activité IFT187;

```
select avg(note) as moy_exa  
      , min(note) as min_exa  
      , max(note) as max_exa  
from Resultat  
where activite = 'IFT187'  
      and te in ('IN', 'FI');
```

- Quelles sont les personnes étudiantes ayant un résultat d'examen entre 70 et 100 ?

```
select *  
from Resultat  
where (te = 'IN' or te = 'FI')  
      and (note between 70 and 100);
```

4. Jointure

```
SELECT opMode { * | projection-extension }  
FROM listeDeJointures
```

```
listeDeJointures ::=  
    denotationTable [ [AS] alias] [ jointure ... ]
```

```
denotationTable ::=  
    nomTable | ( requête )
```

```
jointure ::=  
    produit | jointure_naturelle | jointure_qualifiée |  
    jointure_externe
```

Exemple — Université

- Quels sont les étudiants ayant obtenu une note supérieure ou égale à 70 ?
afficher le matricule, le sigle de l'activité, le titre de l'activité et la note.

```
SELECT R.matricule,  
       R.activite,  
       A.titre,  
       R.note  
FROM (  
    SELECT *  
    FROM Resultat  
    WHERE note >= 70  
    ) AS R  
    JOIN Activite AS A  
    ON R.activite = A.sigle;
```

```
SELECT R.matricule,  
       R.activite,  
       A.titre,  
       R.note  
FROM Resultat AS R  
     JOIN Activite AS A  
         ON R.activite = A.sigle  
WHERE R.note >= 70;
```

4.1. Produit

```
produit ::=  
  {CROSS JOIN | , } denotationTable [ [ AS ] alias ]
```

Exemple — Université

- Afficher toutes les combinaisons possibles entre les étudiants et les activités. Pour chaque combinaison, afficher le matricule et le nom de l'étudiant ainsi que le sigle et le titre de l'activité.

```
SELECT E.matricule,  
       E.nom,  
       A.sigle,  
       A.titre  
FROM Etudiant AS E  
CROSS JOIN Activite AS A;
```

4.2. Jointure naturelle

```
jointure_naturelle ::=  
    NATURAL [ INNER ] JOIN denotationTable [ [ AS ] alias ]
```

La jointure naturelle opère sur tous les attributs de même nom et élimine (automatiquement) la redondance des attributs et leurs résultats.

Exemple — Université

- Produire la liste des étudiants pour lesquels un résultat existe, avec leur matricule, leur nom, l'activité et la note.

```
SELECT matricule,  
       nom,  
       activite,  
       note  
FROM Etudiant  
NATURAL JOIN Resultat;
```

4.3. Jointure qualifiée

```
jointure_qualifiée ::=  
  [ INNER ] JOIN denotationTable [ [ AS ] alias ] qualification  
  
qualification ::=  
  ON conditionJointure | USING ( listeNomCol )
```

- ON $a_1=b_1$ AND ... AND $a_n=b_n$ lorsque les identifiants des attributs de jointure ne sont pas les mêmes (on désigne souvent cette jointure comme étant conditionnelle).
- USING ($a_1, \dots, a_n$) lorsque les identifiants des attributs de jointure sont les mêmes dans les deux tables (on désigne souvent cette jointure comme étant restreinte).

Exemple — Université

- Produire le bulletin d'Éliane. Donner le matricule, le nom, le sigle de l'activité, le nom de l'activité, le trimestre, le type d'évaluation et la note.

```
select sigle, titre, trimestre, te, note  
from Resultat  
  join Activite on (Resultat.activite = Activite.sigle)  
where matricule = '15112354';
```

4.4. Jointure externe

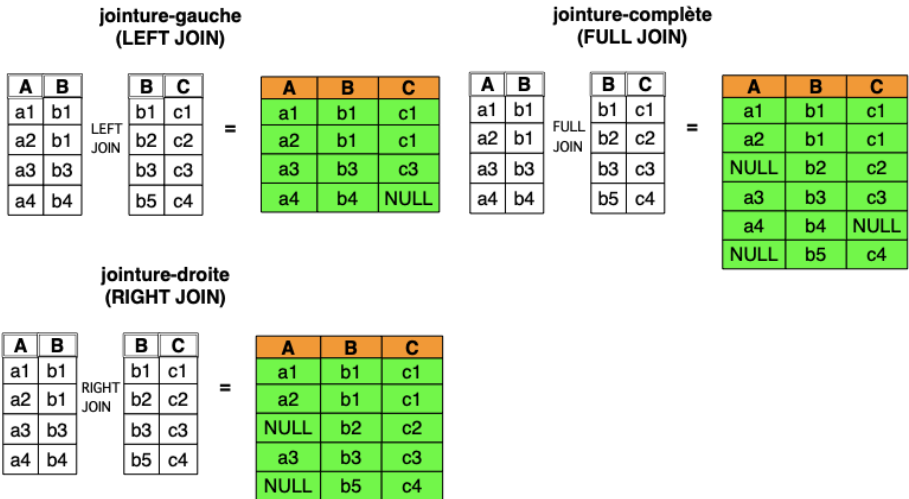


Figure 1. Illustration des jointures externes

```
jointure_externe ::=  
  jExterne JOIN denotationTable [ [ AS ] alias ] qualification  
  
jExterne ::=  
  LEFT [OUTER]   |   RIGHT [OUTER]   |   FULL [OUTER]
```

Une jointure externe permet de ne pas « perdre » les données des tuples sans correspondant en affectant des NULL aux attributs de valeur inconnue.

Le mot OUTER est superfétatoire et son utilisation non recommandée.

Exemple — Université

- Quelles sont les personnes étudiantes n'ayant aucune évaluation à ce jour ?
Donner le matricule et le nom de la personne.

```
select matricule, nom  
from Etudiant  
  left join Resultat using (matricule)  
where activite is null;
```

- Quelles sont les activités n'ayant aucune évaluation à ce jour ?
Donner le sigle et le titre de l'activité.

```
select sigle, titre  
from Activite  
  left join Resultat on (Activite.sigle = Resultat.activite)  
where Resultat.activite is null;
```

5. Opérations complémentaires

```
OpComplémentaire ::=  
  INTERSECT opMode requête  
  | UNION    opMode requête  
  | EXCEPT opMode requête
```

- UNION : rassemble les résultats des deux requêtes
- INTERSECT : conserve seulement les lignes présentes dans les deux résultats
- EXCEPT : conserve les lignes de la première requête qui ne sont pas présentes dans la deuxième

Exemple — Université

- Quelles sont les activités ayant au moins une évaluation à ce jour ?

```
select sigle  
from Activite  
intersect  
select activite  
from Resultat;
```

- Quelles sont les activités n'ayant aucune évaluation à ce jour ?

```
select sigle  
from Activite  
except  
select activite  
from Resultat;
```

- Quelles sont les personnes étudiantes qui ne sont inscrites à aucune activité?

```
select matricule  
from Etudiant  
except  
select matricule  
from Resultat;
```

- Quelle est la moyenne des notes pour IFT187 et IFT159 ?

```
select 'IFT187', round(avg(note), 2) as moy  
from Resultat  
where activite = 'IFT187'  
union  
select 'IFT159', round(avg(note), 2) as moy  
from Resultat  
where activite = 'IFT159';
```

UNIVERSITÉ

Étudiant clé (matricule)

matricule	nom	adresse
15113150	Paul	>Δ ^ς σ> ^ς b
15112354	Éliane	Blanc-Sablon
15113870	Mohamed	Tadoussac
15110132	Sergeï	Chandler

Activité clé (sigle)

sigle	titre
IFT159	Analyse et programmation
IFT187	Éléments de bases de données
IMN117	Acquisition des médias numériques
IGE401	Gestion de projets
GMQ103	Géopositionnement

TypeEvaluation clé(code)

code	description
IN	Examen intra
FI	Examen final
TP	Travail pratique
PR	Projet

Résultat clé(matricule, te, activite, trimestre)

matricule	TE	activité	trimestre	note
15113150	TP	IFT187	20132	80
15112354	FI	IFT187	20122	78
15113150	TP	IFT159	20132	75
15112354	FI	GMQ103	20122	85
15110132	IN	IMN117	20123	90
15110132	IN	IFT187	20133	45
15112354	FI	IFT159	20123	52

Figure 2. Exemple — Université

Conclusion

Références

[Elmasri2016]

Ramez ELMASRI et Shamkant B. NAVATHE;
Fundamentals of database systems;
7th Edition, Pearson, Hoboken (NJ, US), 2016;
ISBN 978-0-13-397077-7.

[Date2015a]

Chris J. DATE;
SQL and relational theory: how to write accurate SQL code;
O'Reilly Media, 2015;
ISBN 978-1-4919-4117-1.

PG

[fr] <https://docs.postgresql.fr/18/index.html> (2026-01-05)

[en] <https://www.postgresql.org/docs/18/index.html> (2026-01-05)

Définitions

SQL (*Structure Query Language*)

Langage de programmation axiomatique fondé sur un modèle inspiré de la théorie relationnelle proposée par E. F. Codd.

[Normes applicables : ISO 9075:2016, ISO 9075:2023]

Produit le 2026-09-23 17:15:31 UTC



Université de Sherbrooke